



Microprocessor Architecture

Terminology

RISC

Reduced instruction set computer - load store model

Turing machine

Read write movable head traversing on a linear tape

Code stream

- Ordered sequence of instructions
- In modern RISC systems the act of moving data between registers and memory is explicitly under the control of the code stream

General instruction types

- Arithmetic instructions - logical operations
- Memory access instructions - load and store operations

Data segment

- Contiguous memory cells that a program stores all of its data in, you only need to know the `base_address` and you can use the formula `base_address + offset` to fetch the full data segment

Memory access

Modern computers allow the contents of a register to be used as a memory address

- Immediate value addressing - register stores the value itself
- Register relative addressing - register stores the `base_address` of value in memory and performs an offset addition operation to load the accurate data segment

Program Execution

Machine code and opcode

→ Machine code is synonymous to machine language

→ Opcode is a term used to describe the mapping of machine language instructions to binary numbers that a computer understands

Binary encoding of arithmetic instructions {requires 2 bytes}

First bit of the instruction is the mode bit = {0, 1} where

- 0 = register type instruction
- 1 = immediate type instructions

Binary encoding of memory instructions {requires 2 bytes}

- Immediate type load - source address is passed as argument
- Register type load - register containing memory address is passed as an argument
- Register relative load - register containing `base_address` and immediate value of offset are passed as arguments

Programming model & ISA

- Programming model is the programmer's interface to the microprocessor
- Instruction fetch - special type of load that happens automatically for every instruction
- Fetch-Execute loop is repeated for as long as the computer is powered on

Branch instructions

- Unconditional branch - essentially a jump, target can be either an immediate value or an address stored in a register
- Conditional branch - conditional jump, requires the presence of **processor status word (PSW)** register which is used to reference the conditional

Branches point the processor to different sections of the code stream by manipulating its control unit which contains the instruction register and program counter - branch instructions can also use register-relative addressing

In the real world, programmer places a label on the line of code to which he wants to jump and then puts the label in the branch's target field

Boot up

- The first instruction which is loaded into the processor's instruction register is the first line of a program called BIOS which is attached to the computer's motherboard
- At the end of the BIOS program lies a jump instruction which targets the location of a boot-loader program whose job is to search and load the operating system from the hard-drive, the operating system in turn loads and unloads all the programs from this point onward

Pipelined Execution

→ Front End roughly corresponds to the control and I/O units

→ Back End roughly corresponds to the ALU and registers

Instruction completion rate = the number of instructions completed per unit of time

→ Non-pipelined instructions are also called single-cycle processors because all instructions take exactly one clock cycle to execute

→ Pipelining is essentially the process of breaking down the processor's operations into stages to enable parallel and independent execution of each

stage, these stages are usually coupled with each other using buffers(queues) — this stage-level parallel execution gets rid of coupled wait times and increases instruction completion rate

pipeline depth = number of stages in the pipeline

→ Clock cycle time is shrunk to match the time it takes each stage to complete its work so that at the start of each clock cycle each pipeline stage hands-off the instruction to next stage

→ New instruction is completed at the close of only those clock cycles in which write stage has been working on the instruction

→ Pipelining doesn't speed up instruction execution time but it does speed up program execution time

→ Pipelined vs Non-pipelined completion rate delta is loosely governed by this formula: -

completion rate delta $\sim$ = number of pipeline stages

→ Good rule of thumb: stages must be of exactly equal duration ~ each stage requiring one clock cycle

program execution time = number of instructions in a program / instruction completion rate

→ Pipelined processor approaches its ideal completion rate if it can go long stretches with its pipeline full on every clock cycle

- Instruction throughput = number of instructions completed in each clock cycle
- Max theoretical instruction throughput = for simple processors, one per cycle
- Average instruction throughput = IPC (instructions per clock) that processor has actually completed

Pipeline Stall

→ Occurs when an instruction gets stuck in one stage for more than a clock cycle {AKA pipeline bubbles}

→ Pipeline stalls decrease processor's average instruction throughput

Instruction latency = number of clock cycles required for an instruction to pass through the pipeline

→ Slowest stage in the pipeline will determine the length of CPU's clock cycle slowing every other stage with it

- Pipeline stalls must be avoided - avoided using caching
- Pipeline refills must be avoided - avoided using decouple queues and buffers

→ Pipelining requires non-trivial amount of extra bookkeeper and buffering logic, incurs overhead cost in transistors and die space

Superscalar Execution

→ Interface represents a sequential execution machine but the processor is actually a parallel execution machine

More parallel ALUs $\approx$ More instructions executed per cycle

→ ALU can be broken down into 2 major units : -

- Execution units
 - Integer execution unit
 - Floating point execution unit
- Memory access units
 - Load store unit
 - Branch execution unit

processor microarchitecture = specific processor's implementation of ISA

→ IBM System/360 abstracted the ISA layer by introduction of microcode engine which consists of microcode ROM which holds microcode programs, all programs are effectively running in emulation

→ RISC axed the microcode engine in favor of less complex instructions directly being executed on the processor without any complex intermediary layer (ISA ex - powerPC, Arm)

→ CISC saw crazy implementations of microcode engines (ISA ex - x86)

Challenges to superscalar design

- Data hazard — intra-stage parallel execution breaks the intended sequence of inter-stage operations, alleviated using 2 methods : -
 - Forwarding — processor takes the o/p from one port and directly feeds it to the ALU i/p port bypassing corruptions arising from register-file write stage
 - Register renaming — breaks parallel execution deadlocks by introducing a shadow temporary register, op1 feeds o/p to this temporary register instead of directly feeding it to an i/p port

- Structural hazards — processor simply doesn't have enough hardware to execute multiple instructions at once
- Control hazards (AKA branch hazards) — arise when processor arrives at a conditional branch and has to decide which instruction to fetch next — instruction load latency is added to branch condition evaluation latency, alleviated using 2 methods : -
 - Branch prediction
 - Instruction caching

Instruction Dispatch

Mainly follows these 4 rules

- In-Order dispatch rule — instructions dispatch from the instruction queue in program order, out of order execution only begins after reservation stations
- Issue buffer or execution unit availability rule
 - when heading to reservation station, there must be an entry available in the reservation station
 - when heading to execution unit, unit's pipeline slot must be empty and its reservation slot must be empty
- Completion buffer availability rule — completion queue keeps track of the program order of each in-flight instruction, so any instruction that enters the out-of-order back end must be logged in the completion queue first
- Rename register availability rule — there must be enough rename registers available to temporarily store the results for each register that instruction will modify

Branch prediction

→ Branch unit still counts as a memory access unit because it works closely with the instruction fetcher, BU contains branch execution unit (BEU) and branch prediction unit (BPU)

→ Modern processors use a technique called speculative execution, which involves making educated guesses before the branch's condition is actually evaluated

2 Types of branch prediction : -

- static branch prediction — simple, relies on assumption that majority of backward-pointing branches occur in the context of repetitive loops
- dynamic branch prediction — involves the use of BHT and BTB to record information about the outcomes of branches that have already been executed
 - BHT (branch history table) creates an entry for each conditional branch the BU has encountered on its last few cycles
 - BTB (branch target buffer) stores the branch targets for previously executed branches, when a branch is executed the BPU grabs the speculative branch target from the BTB

→ BTIC (branch target instruction cache) — stores the instruction that is stored at the branch's target address, this removes the requirement to go through code storage to fetch the instructions corresponding to the entry in BTB

SIMD / Vector computing

SIMD = single instructions, multiple data AKA vector computing

→ Processors are SISD (single instruction, single data) devices which are not good for multi-media computing

→ SIMD introduces data parallelism

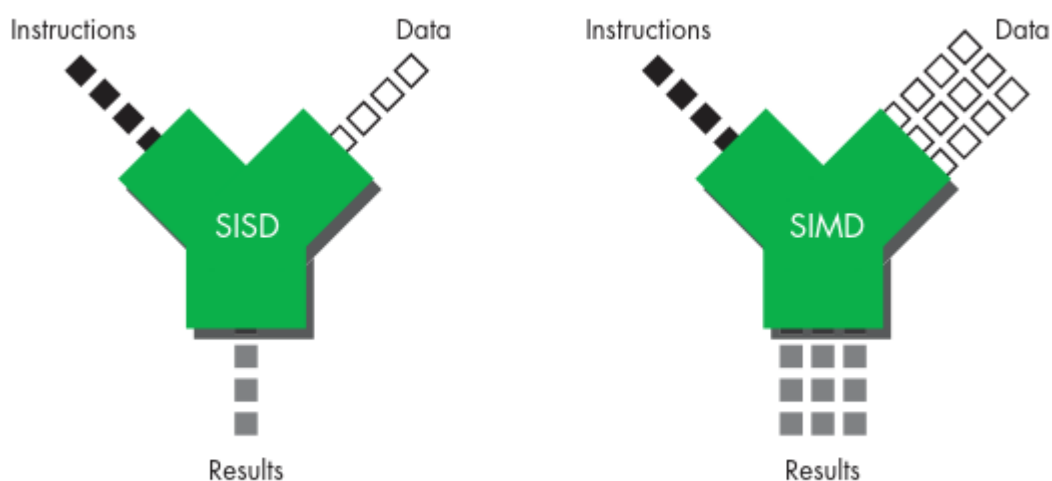


Figure 8-1: SISD versus SIMD

→ A superscalar CPU operates on multiple scalars at once, but it performs a different operation on each instruction. A vector processor lines up a whole row of scalars, all of the same type, and operates on them in parallel as a unit

→ Vectors are represented in what is called a packed data format, in which data are grouped into bytes or words and packed into a vector of a certain length

4 major types of vector operations : -

- intra-element arithmetic operations (mathematical operations on elements of different vectors)
- intra-element non-arithmetic operations (logical operations on elements of different vectors)
- inter-element arithmetic operations (mathematical operations on elements of same vector)
- inter-element non-arithmetic operations (logical operations on elements of same vector)

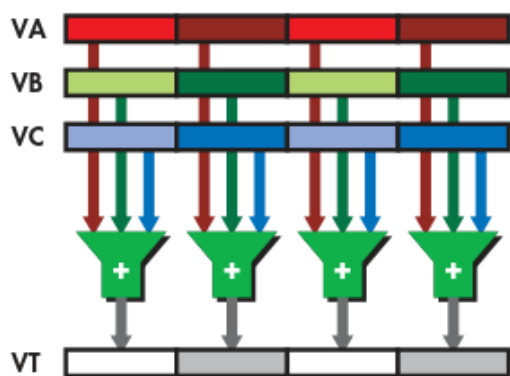


Figure 8-4: Intra-element arithmetic operations



Figure 8-5: An inter-element sum across operation

Design Philosophies

→ Deeper pipelining increases instruction execution time, this can damage instruction completion rate if the pipeline has to be flushed and refilled often

Motorola G4e

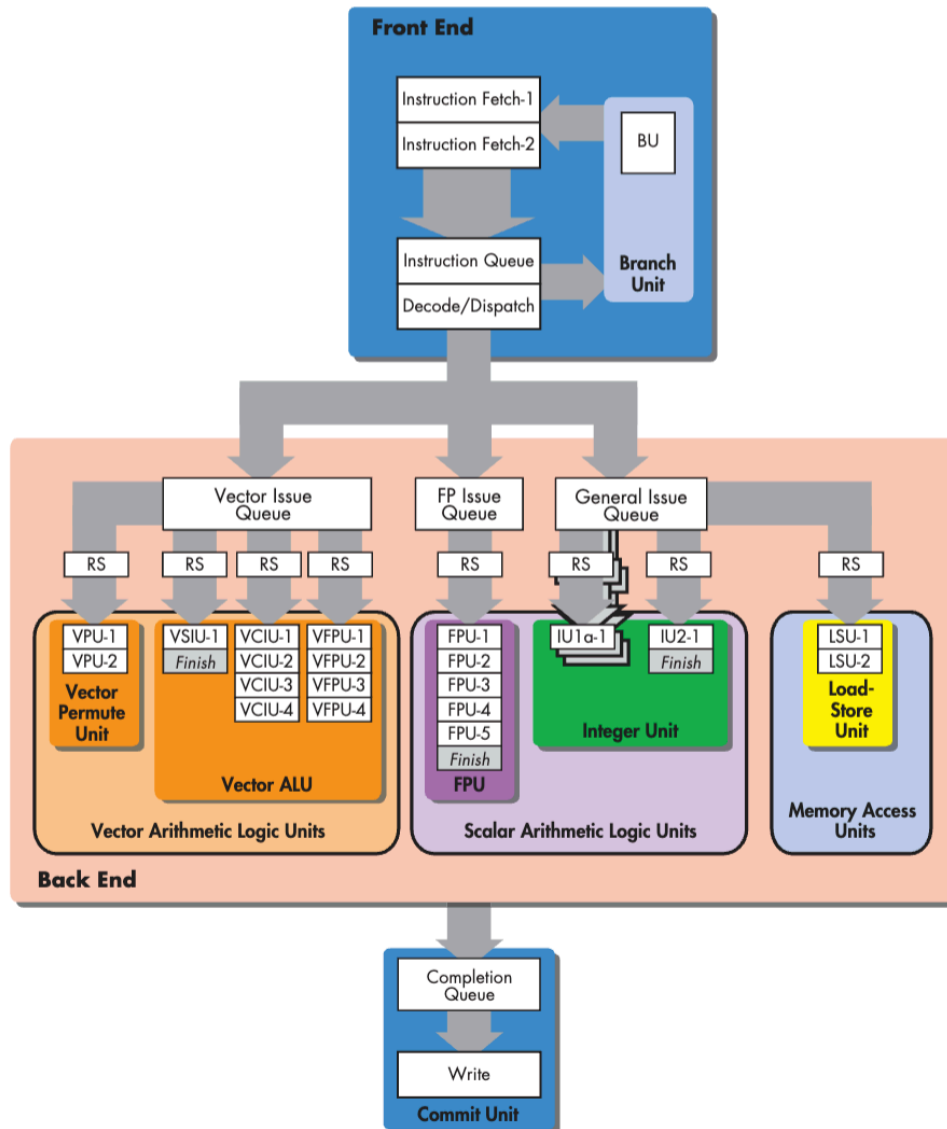


Figure 7-4: The basic microarchitecture of the G4e

- Focuses on getting a small number of instructions onto the chip at once and spreading them widely to execute in parallel in as few cycles as possible
- Shallow and wide approach
- G4e L1 cache = 32KB instruction cache + 32KB data cache = 64KB
 - instructions can enter G4e pipeline after going through the instruction cache
- Issue Queues and their types : -
 - General instruction queue — feeds the integer ALU — order is not important — lac of order allows memory and code instructions to flow

around instructions stalled in execution phase

- Vector instruction queue — feeds the vector execution units — in-order — feeds from bottom only
- Floating point issue queue — feeds to the floating point execution unit

Pentium 4

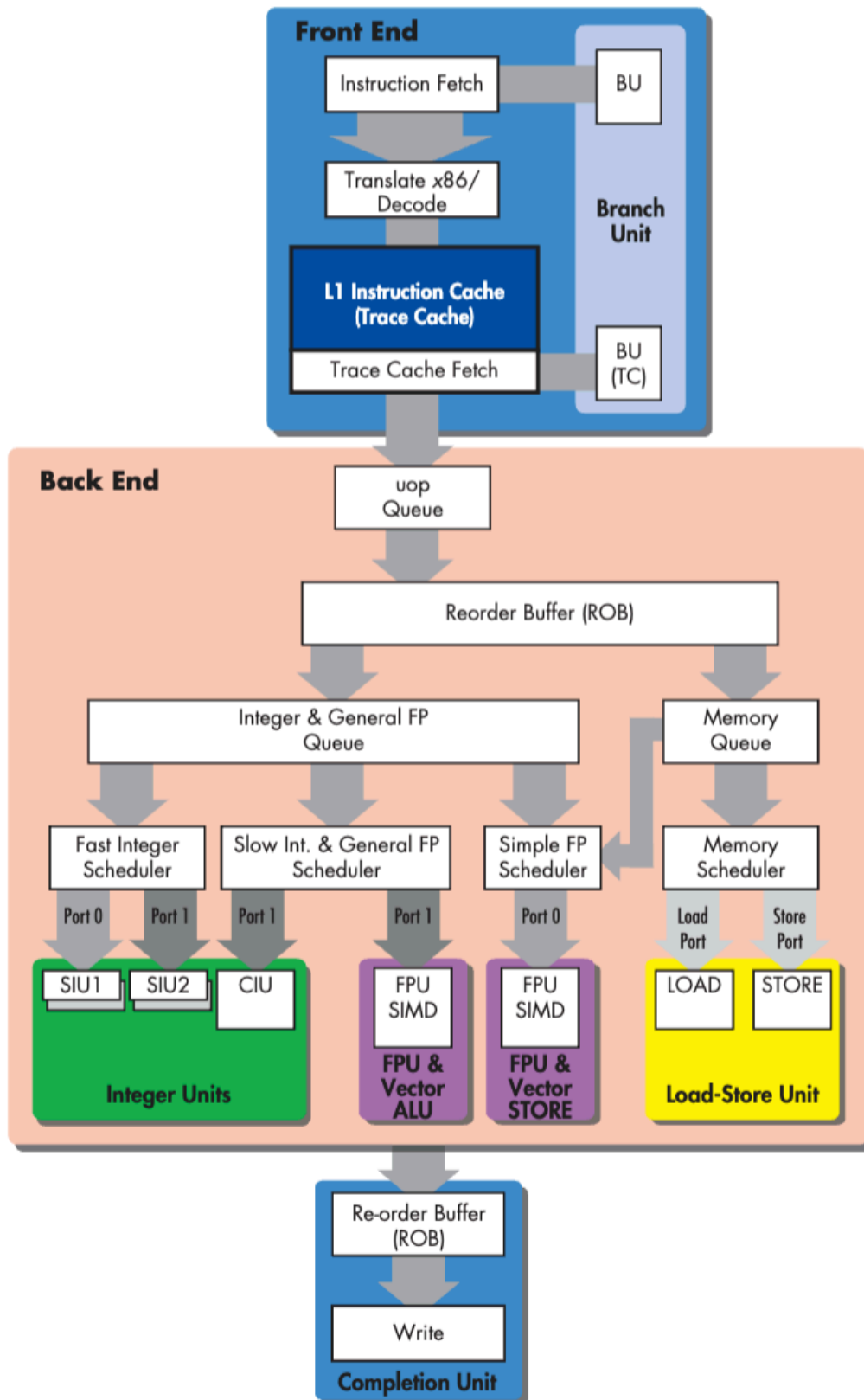


Figure 7-5: Basic architecture of the Pentium 4

→ Narrow and deep approach

→ Decoupled back-end from front-end using reservation stations (instruction pools)

Trace cache

→ Converted L1 cache to trace cache for decoded micro-ops

L2 cache → decode into strings of micro-ops called traces → micro-ops are fit into trace cache

→ Pentium 4 executes these traces instead of x86 code produced by the compiler

→ trace cache hit > 90% of times

→ When there's an L1 cache miss, trace cache goes into trace segment build mode where front-end fetches x86 code from L2 cache, translates it into micro-ops, builds a trace segment with it and loads that segment into trace cache to be executed

→ If you have a chunk of x86 code with branch in it, the trace cache builds a trace from the instructions upto and including the branch instruction, then it picks which branch the code will take and continues building the trace along that speculative branch

→ Pentium 4's trace cache implements branch folding

→ Trace cache inserts into the trace segment a tag that points to the section of the microcode ROM containing the micro-op sequence for that particular instruction

→ Trace cache has its own small little BPU (branch prediction unit)

→ Drive stage : pentium 4 is so fast that sometimes a signal can't make it all the way to where it needs to be in a single clock pulse, so the processor dedicates some pipeline stages to let these signals propagate across the chip

→ Pentium 4's micro-op queues are FIFO queues, there are 2 queues which are memory micro-op queue and arithmetic micro-op queue

Simple pipeline stage flow

Trace cache next instruction pointer → trace cache fetch → drive → allocate and rename (ROB) → queue → schedule → issue → register files → execute → flags → branch check → drive → complete and commit

x86_64

Terminology

EPIC = Explicitly Parallel Instruction Computing is a mix of very long instruction word ISA, prediction, speculative loading and other compiler-oriented, performance-enhancing techniques

- AMD introduced 64 bit extensions to the x86 ISA and named it x86_64
- The labels 64bit, 32bit, etc are used to denote the number of bits that each of processor's general purpose registers (GPR) can hold
- Overflow : result is greater than the highest positive integer
- Underflow : result is less than the largest negative integer

Improvements in x86_64 over x86

- Program counter and data stream doubled in size, but instruction stream stayed the same
- n bits allow you to represent 2^n integers in binary
- x86_64's physical address space is 40 bits wide — supports 282TB of virtual address space and 1TB of RAM
- Because memory address values are twice as large, they take twice the cache size
- Doubled the sizes of all general purpose registers
- Number of SIMD registers and GPRs available to programmers has gone up to 16 each
- introduced mode switching
 - Long mode
 - 64 bit mode — 64 bit OS and 64 bit code
 - Compatibility mode — extended registers are not available to the program
 - Legacy mode — 32 bit OS and 32 bit code
- Default data size for integer instructions is still 32 bits — this is to save cache space — if you wish to use a larger or smaller integer you must add a one byte length prefix (AMD calls it a REX prefix - register extension)

Cache

L1 cache

- Sits closest to the processor
- Made out of SRAM (4 to 6 transistors, instead of 1 transistor in DRAM)

L2 cache

- L2 usually contains all the data in L1 and some more
- L1 is a subset of L2, generally speaking - each level in the hierarchy is a subset of the level above it

Cache miss

- Compulsory miss — barring the use of data-prefetching, its the one type of miss that can't be avoided, all cached data must be brought into the cache for the very first time
- Conflict miss — multiple cache blocks require the same frame, this causes a conflict
- Capacity miss — a block containing needed data has been evicted from the cache due to a lack of capacity

Locality of reference

- Spatial locality — if a CPU needs an item from memory at any given moment, its likely to need that item's neighbors next
- Temporal locality — if an item in memory was accessed once its likely to be accessed again in near future

→ Media applications have excellent temporal locality for code but poor temporal locality for data — this gave rise to cache split, I cache for instructions and D cache for data

Cache pollution

Application fills up the cache with data that doesn't really need to be cached because it won't be used again — sins of spatial locality — think mp3 apps

Cache Organization

- The actual piece of requested data is called critical word and surrounding data is called cache line or cache block
- When a block is moved from RAM to cache, it is placed in a special slot in cache called a block frame
- Cache placement policy governs where in the cache a block from memory can be placed

Tag RAM

- Made out of SRAM, holds information about blocks currently being stored in the frame, used to determine : -
 - cache hit or cache miss
 - location of block within the cache
 - location of critical word within the cache
- Tag RAM can add unwanted access latency

Fully associative mapping

- Any RAM block can be stored in any available block frame

Direct mapping

- Each block frame can cache only a certain subset of the blocks

Way Set associative mapping

- Restrict the caching of main memory blocks to a subset of the available cache frames
- n way set associative mapping : cache is divided into sets of n frames each, decreased odds of collision and low access latency
- higher the value of n, higher the cache latency
- A direct map set is simply a 1 way set associative map
- A fully associative set is an n-way set associative map

Eviction policies

- With simple policies like FIFO and LIFO you wind up evicting blocks that will be used again shortly, thereby increasing cache miss rate

→ Most caches implement some kind of pseudo-LRU (least recently used) algorithm that approximates true LRU by marking blocks as dirty the longer they sit unused in the cache

Block sizes

→ If you increase the block size while keeping the cache size the same, you decrease the number of blocks that the cache can hold. Fewer blocks in the cache means fewer sets, fewer sets means more collisions and more misses

→ Large cache blocks are more prone to pollute the cache with non-reusable data than small cache blocks

→ Larger the block size, the more data is fetched with each load, so large block sizes can really eat up memory bus bandwidth, especially if the miss rate is high

Write policies

- Write through — a piece of modified data would be written to both the L1 and main memory so that all of its copies are current. Writes the modified data through the entire hierarchy
- Write back — results in less memory traffic, changes propagate down to the lower levels of the hierarchy as cache blocks are evicted from the higher levels